

A Guided Tour of the Theory for the Problem-Solving Model

Three contributions — the logic, the calculations, and a critical reading

Working note — prepared for Fausto

June 16, 2026

Abstract

This note walks through the three rounds of analytical work on our agent-based model: Atiyab’s first “self-averaging” mean-field, Atiyab’s second note (a dynamical rate equation and an *exact* pair-level framework), and Juan’s recent “beyond mean-field” steady-state calculation. It is written for a reader who knows the *model* but not theoretical-physics derivations, so every symbol is defined and the *logic and the calculation behind every step* are spelled out, with a coloured box at each step saying *why* we are doing it. It is also deliberately *critical*: after each contribution I state what it gets right, what it gets wrong, and — crucially — I show that Juan’s central calculation rests on a modelling error (treating the greedy step as flipping *two* variables at once, when the algorithm flips *one* at a time). I then derive the corrected single-variable version and show, against fresh simulation measurements, that the correction moves the prediction from $\bar{p} \approx 0.66$ to $\bar{p} \approx 0.78$, much closer to the measured 0.74. All numbers in this note were measured or computed by the accompanying scripts (`_check_juan.py`, `_corrected_mf.py`).

Contents

1	What we predict, and how the model actually updates	1
2	Atiyab #1: the simple “self-averaging” mean-field	2
2.1	The hypothesis	2
2.2	Calculation: the violation fraction $V(p)$	2
2.3	Calculation: the best p	3
2.4	Critical assessment	3
3	Atiyab #2: dynamics, and an exact pair-level framework	3
3.1	Half A — the attractor as a flow (this half is single-variable, and correct in structure)	3
3.2	Half A — a rate equation and a timescale	4
3.3	Half A — the honest admission	4
3.4	Half B — the exact pair-level framework	4
3.5	Half B — exact single-variable flip rules (verified in code)	4
3.6	Half B — where it stops, and why	5
4	Juan: computing the steady state “beyond mean field”	5
4.1	What Juan does (faithful walk-through)	5
4.2	The central error: the greedy does not flip two variables at once	5
4.3	The corrected single-variable calculation	6
4.4	A second, independent problem: V is computed with the discredited formula	6
4.5	Smaller criticisms worth raising on the call	7
4.6	What is nonetheless right and valuable in Juan’s note	7

5	Synthesis and the real frontier	7
5.1	The through-line	7
5.2	Two concrete next steps	7
A	Reproducibility	8

1 What we predict, and how the model actually updates

A *simulation* tells us *what* happens; a *theory* tries to tell us *why*, and ideally hands us a formula that predicts the measured numbers without running the code. The three quantities the theory targets:

- V — the **violation count**: how many of the $M = \alpha K$ universal clauses an agent’s assignment violates (lower is better). We will mostly use the *fraction* $V/M \in [0, 1]$.
- p — the **fraction of ones**: across all bits in the system, the fraction set to 1.
- H — the **homogeneity**: how much agents agree, variable by variable.

The two clause types (used constantly). Every clause has two variables. AND is satisfied only when *both* bits are 1; XOR is satisfied only when the bits *differ* (exactly one 1). If the two bits are equal, XOR is violated. Half the clauses are AND, half XOR.

The target numbers (measured, not assumed). So we can judge each theory against reality, here is what the simulation actually does in steady state (Random network, $K = 30$, $N = 80$, $p_{\text{obs}} = 0.01$, $\tau = 10$; averaged over the last 2000 of 4000 ticks, 3 seeds; `_check_juan.py`):

α	sim p	sim V/M	note
2	0.708	0.491	V/M just below 0.5
4	0.739	0.487	
8	0.744	0.466	p saturating ≈ 0.74 ; V/M falling <i>below</i> 0.5

Keep two facts from this table in mind, because every theory below will be judged on them: **(i)** p settles around 0.7–0.74 and *saturates* (it does not climb toward 1); **(ii)** the violation fraction is *below* 0.5 and falling.

How the model updates (this detail decides everything later). When an agent learns a new clause $c = (i, j)$, the routine `local_update_around` does the following: it takes the *two* variables of c , shuffles their order, and then considers them **one at a time**. For each variable it tentatively flips *that single bit* and keeps the flip if and only if it *strictly* lowers the violation count on the clauses sharing a variable with c . It **never** evaluates flipping both bits together.

Why we do this. This last paragraph looks like a technicality, but it is the single most important fact in this note. Two of the three contributions get the dynamics right precisely because they respect “one variable at a time”; Juan’s steady-state calculation goes wrong precisely because it does not. We flag it now so the error is visible when we reach it.

In plain terms. Everything below rests on one trick: instead of tracking all $N \times K$ bits, summarise the system by a few averages (like p) and write equations they must obey. That is a *mean-field approximation*. The three contributions are increasingly careful versions of this trick — and the interesting failures happen exactly where the averaging hides something real.

2 Atiyab #1: the simple “self-averaging” mean-field

Source: Self Average Approximation.pdf.

Why we do this. We start here because it is the simplest possible theory and it fixes the vocabulary (p , the violation formula). Even though it turns out to be wrong about *where* the system settles, it is right about the *level* of violations and about network-independence, so it is the natural baseline every later theory must beat.

2.1 The hypothesis

After a long run, each agent has exchanged so many clauses that its knowledge base looks like a *random sample of the global clause pool*; and since all agents sample the same pool, their bits should settle into the same statistics. So:

Every bit is 1 with the same probability p , independently of every other bit.

In plain terms. This “self-averaging” assumption throws away *who* talks to *whom*, *which* agent holds *which* clause, and — the part that will come back to bite us — any *correlation* between the two bits of a clause. The whole system is compressed to one number p .

2.2 Calculation: the violation fraction $V(p)$

Step 1 (one clause). If each bit is 1 with probability p and bits are independent:

$$\Pr(\text{AND VIOLATED}) = 1 - \Pr(\text{both 1}) = 1 - p^2, \quad (1)$$

$$\Pr(\text{XOR VIOLATED}) = \Pr(\text{both equal}) = p^2 + (1 - p)^2. \quad (2)$$

Step 2 (all M clauses, half each type). Multiply each probability by $M/2$ and add:

$$V(p) = \frac{M}{2}(1 - p^2) + \frac{M}{2}(p^2 + (1 - p)^2) = M\left(1 - p + \frac{p^2}{2}\right). \quad (3)$$

(The algebra: the $\pm \frac{M}{2}p^2$ terms cancel, leaving $\frac{M}{2}(1 + (1 - p)^2) = M(1 - p + p^2/2)$.)

2.3 Calculation: the best p

Why we do this. We want the p that the system should prefer. “Prefer” means “fewest violations,” i.e. the bottom of the curve $V(p)$. The slope (derivative) is zero at the bottom, so we set it to zero.

$$\frac{dV}{dp} = M(p - 1) = 0 \Rightarrow p^* = 1, \quad \frac{d^2V}{dp^2} = M > 0 \text{ (a valley)}, \quad \boxed{V^* = \frac{M}{2} = \frac{\alpha K}{2}}. \quad (4)$$

As a fraction, $V^*/M = 1/2$.

2.4 Critical assessment

Critical note. The theory predicts $p^* = 1$: *every* agent should drive *every* bit to 1. The simulation says $p \approx 0.7$ (table in §1). So the headline conclusion is simply false. It *looks* successful only because of a coincidence: $V(p)$ in (3) is very flat near its bottom, so being wrong about p (0.7 vs 1) costs little in V ($0.545M$ vs $0.5M$, about 9%). **The theory is right about the level for the wrong reason**, and a critical reader should say so rather than treat the $\sim 10\%$ agreement as a success. It also fails outright when the observation probability or clause interval is varied — exactly the regimes where “independent bits” is worst.

Two things it does get genuinely right, and which survive into the final story:

1. **Network-independence of the long-run level.** Because `comm_scale` normalises incoming communication by average in-degree, every agent receives the same *average* information rate regardless of topology; and once all knowledge bases are samples of the same pool, the greedy step sees the same statistics everywhere. Topology changes *who* transmits and *how fast*, not the destination.
2. **The level $\alpha K/2$ is actually a good predictor** (the measured $V/M \approx 0.49$ at $\alpha = 2$ is within 2% of 0.5). Ironically this trivial formula predicts V *better* than Juan’s later refinement does — a point we return to in §4.

3 Atiyab #2: dynamics, and an exact pair-level framework

Source: Mean Field Model_version1.pdf. Two halves: first it makes the theory *dynamical* (how fast, not just where); then it builds an *exact* replacement for the crude independence assumption.

3.1 Half A — the attractor as a flow (this half is single-variable, and correct in structure)

Why we do this. Minimising a curve tells you the destination but not the route. To get convergence *speed* — which is what the paper ultimately cares about — we need to watch what the greedy step does to a single bit, over and over.

Consider testing a flip of one variable j from $0 \rightarrow 1$. A neighbouring clause containing j changes status according to its *partner* bit:

Clause	partner = 1 (prob. p)	partner = 0 (prob. $1 - p$)
AND	01 $\rightarrow$ 11: violated $\rightarrow$ satisfied, $\Delta = -1$	00 $\rightarrow$ 10: stays violated, $\Delta = 0$
XOR	01 $\rightarrow$ 11: satisfied $\rightarrow$ violated, $\Delta = +1$	00 $\rightarrow$ 10: violated $\rightarrow$ satisfied, $\Delta = -1$

Averaging over the partner: $\mathbb{E}_{\text{AND}}[\Delta] = p(-1) + (1-p)(0) = -p$ and $\mathbb{E}_{\text{XOR}}[\Delta] = p(+1) + (1-p)(-1) = 2p - 1$. Each variable sits in $\sim \alpha$ AND and $\sim \alpha$ XOR clauses, so the total expected change of flipping one zero up is

$$\Delta_{0 \rightarrow 1} \approx \alpha(-p) + \alpha(2p - 1) = -\alpha(1 - p) \leq 0, \quad (5)$$

and symmetrically $\Delta_{1 \rightarrow 0} \approx +\alpha(1 - p) \geq 0$. So on average up-flips help and down-flips hurt: the system drifts towards $p = 1$.

Critical note. Note this half is honest about the mechanism: it flips *one* variable and sums over *that variable’s* clauses. That is exactly what the code does. The flaw here is only that it reads “the average is negative” as “the flip is always accepted” — ignoring that the *realised* change is random and is filtered by the greedy rule. That filtering is what later pulls the true attractor below 1.

3.2 Half A — a rate equation and a timescale

Why we do this. A *rate equation* is “rate of change = how often something happens $\times$ how much it changes things.” It converts the drift above into a prediction of convergence *speed*.

Only $0 \rightarrow 1$ flips change p , and they are available in proportion to the fraction of zeros $(1 - p)$. With learning rate r :

$$\frac{dp}{dt} = \frac{2\gamma}{K} r (1 - p) \Rightarrow p(t) = 1 - (1 - p_0)e^{-(2\gamma r/K)t}, \quad \tau = \frac{K}{2\gamma r}. \quad (6)$$

Crucially $r = r_{\text{obs}} + r_{\text{int}}$ with $r_{\text{int}} \propto$ average in-degree: **the network enters here, in the timescale** — speed, not destination, as #1 predicted.

Critical note. The whole dynamic half rides on the false $p = 1$ attractor, so the *endpoint* of $p(t)$ is wrong. But the *shape* (exponential relaxation) and the key qualitative claim (network sets the timescale via r) are sound and are the seed of the paper’s transient story.

3.3 Half A — the honest admission

The note then states plainly that $p = 1$ is *not* the real attractor (the simulation converges below 1), so $H = \max(p, 1 - p)$ cannot be the full story. Hence: “we need a better mean-field.”

3.4 Half B — the exact pair-level framework

Why we do this. The crude step was treating the two bits of a clause as independent ($\Pr(\text{both } 1) = p^2$). But the greedy step *couples* them, so they are correlated and p^2 is wrong. The fix is to stop factorising and track the joint distribution of *pairs* of bits.

For a clause on (i, j) define $q_{ab}^{(ij)} = \Pr(x_i = a, x_j = b)$. Then the violation probabilities are *exact*: $v_{\text{AND}} = 1 - q_{11}^{(ij)}$ and $v_{\text{XOR}} = q_{00}^{(ij)} + q_{11}^{(ij)}$. Averaging over all AND clauses (M_A) and XOR clauses (M_X) gives an **exact identity**:

$$V(t) = M_A(1 - q_{11}^A(t)) + M_X q_{\text{eq}}^X(t). \quad (7)$$

In plain terms. “Exact identity” means *no approximation*: if you knew the pair quantities, this would give the true V . The catch is getting them.

3.5 Half B — exact single-variable flip rules (verified in code)

Why we do this. To get the dynamics of those pair quantities we need to know exactly when a flip is accepted. Atiyab does this honestly — and, importantly, *one variable at a time*, matching the code.

For variable j , let A_1, A_0 be the number of AND clauses on j with partner 1, 0, and X_1, X_0 the XOR clauses with partner 1, 0. Counting the violation change of a *single*-bit flip gives the exact, code-verified rules:

$$\text{flip } 0 \rightarrow 1 \text{ accepted} \iff A_1 + X_0 > X_1, \quad (8)$$

$$\text{flip } 1 \rightarrow 0 \text{ accepted} \iff X_1 > A_1 + X_0. \quad (9)$$

Atiyab checked these against a real run: *every* accepted flip obeys the inequality (100% match). **Remember equations (8)–(9): they are the correct acceptance rules, and Juan’s calculation should have been built on them.**

3.6 Half B — where it stops, and why

Differentiating (7) gives dV/dt in terms of the pair quantities; but the rate of change of *pairs* depends on *triples* (flipping j disturbs every pair attached to j), triples depend on quadruples, and so on.

In plain terms. This is the system being “*not closed*”: the equation for what you want depends on a more complicated thing you don’t have. The pair framework is therefore *correct but not yet solvable* — an honest, exact scaffold with a gap in the middle.

If you *force* the factorisation $q_{11}^A \approx p^2$, $q_{\text{eq}}^X \approx p^2 + (1 - p)^2$, (7) collapses back to (3). So #2 is a strict *generalisation* of #1, not a rival.

4 Juan: computing the steady state “beyond mean field”

Source: `clauses.pdf` (email “Notes beyond mean field”, 16 June 2026).

Why we do this. Juan attacks the one thing #1 got wrong — the claim $p^* = 1$ — by *not assuming* where the system settles, and instead computing it from how often the greedy step *accepts* flips. This is exactly the right instinct.

4.1 What Juan does (faithful walk-through)

Step 1 — a rate equation from pair states. When a clause lands on a variable pair, that pair is 00, 10, 11 with probabilities $(1-p)^2$, $2p(1-p)$, p^2 . Let F_{00} be the probability the greedy *accepts* flipping a 00 pair up to 11 (adds two ones), F_{11} for $11 \rightarrow 00$ (removes two ones); a 10 pair flip leaves the count unchanged. Then

$$\frac{dp}{dt} = 2(1-p)^2 F_{00} - 2p^2 F_{11}. \quad (10)$$

Step 2 — steady state. Set (10) to zero. The algebra: $2(1-p)^2 F_{00} = 2p^2 F_{11} \Rightarrow \frac{(1-p)^2}{p^2} = \frac{F_{11}}{F_{00}} \Rightarrow \frac{1-p}{p} = \sqrt{\frac{F_{11}}{F_{00}}}$, hence

$$\bar{p} = \frac{\sqrt{F_{00}}}{\sqrt{F_{00}} + \sqrt{F_{11}}}. \quad (11)$$

Step 3 — violations. Using the *same independent-bit formula* as everyone, $V/M = \frac{1}{2}(1-\bar{p}^2) + \frac{1}{2}(\bar{p}^2 + (1-\bar{p})^2) = \frac{1}{2} + \frac{1}{2}(1-\bar{p})^2$. **Step 4 — the accept rates.** For a $00 \rightarrow 11$ flip he counts the violations created/removed among neighbouring clauses (means $\mathbb{E}[N^+] = 2\alpha p$, $\mathbb{E}[N^-] = 2\alpha$), models them as Poisson, takes the difference (a Skellam) $Z_{00} = \text{Poisson}(2\alpha p) - \text{Poisson}(2\alpha)$, and assembles F_{00}, F_{11} from the probabilities $H_{00}(k) = \Pr(Z_{00} \leq k)$. **Step 5.** Substituting F_{00}/F_{11} into (11) gives an implicit equation; solving numerically yields $\bar{p} \approx 0.66$ at $\alpha = 4$ and $\bar{p} \rightarrow 1$ as $\alpha \rightarrow \infty$.

4.2 The central error: the greedy does not flip two variables at once

Critical note. Juan’s whole construction treats the move as “the pair (0,0) flips *together* to (1,1),” accepted with one probability F_{00} . But the algorithm (`local_update_around`) flips **one variable at a time** (§1), and Atiyab’s Half B already wrote down the correct single-variable acceptance rules (8)–(9). So F_{00} is the acceptance probability of a joint two-bit move the model *never performs*. This single mistake propagates everywhere:

1. **The squared prefactors are spurious.** Requiring *both* bits to be 0 before an up-move is what puts $(1-p)^2$ in (10); that square is exactly what becomes the square root in (11). With single-bit flips there is no such requirement.
2. **It ignores the (1,0) outcomes.** In reality, on a (0,0) clause the greedy may accept flipping one bit and reject the other, landing at (1,0) — which *does* change p . Juan discards 10 pairs as “no change”; for single-bit dynamics they are a live channel.
3. **It uses the wrong acceptance probabilities.** The correct objects are $\Pr(\text{flip } 0 \rightarrow 1 \text{ accepted})$ and $\Pr(1 \rightarrow 0)$ from (8)–(9), not a joint F_{00}, F_{11} .

4.3 The corrected single-variable calculation

Why we do this. We redo Step 1–2 honestly: one variable at a time, using Atiyab’s verified rules. This is what Juan’s calculation should have been.

Focus on one variable j . With $A_1 \sim \text{Bin}(\alpha, p)$, $X_1 \sim \text{Bin}(\alpha, p)$, $X_0 \sim \text{Bin}(\alpha, 1-p)$ (each variable is in α AND and α XOR clauses, partners independently 1 w.p. p), define $S = A_1 + X_0 - X_1$. By (8)–(9),

$$F_{\text{up}} = \Pr(S > 0), \quad F_{\text{dn}} = \Pr(S < 0). \quad (12)$$

Per learning event the two clause variables are each considered once; a fraction $(1 - p)$ of them are 0 (each may flip up, +1 one) and p are 1 (each may flip down, -1 one). So the *single-variable* rate equation is **linear**, not squared:

$$\frac{dp}{dt} \propto (1 - p)F_{\text{up}} - p F_{\text{dn}} \Rightarrow \boxed{\bar{p} = \frac{F_{\text{up}}}{F_{\text{up}} + F_{\text{dn}}}}. \quad (13)$$

No square root. Solving (13) self-consistently (`_corrected_mf.py`, exact enumeration of the binomials) gives:

α	Juan (11)	corrected (13)	simulation
2	—	0.748	0.708
4	0.66	0.784	0.739
8	—	0.822	0.744

In plain terms. The correction is not cosmetic. Juan’s $\bar{p} = 0.66$ *undershoots* the true 0.739; the single-variable version gives 0.784, roughly halving the error and landing on the right scale. The double-flip assumption was actively dragging the prediction down.

4.4 A second, independent problem: V is computed with the discredited formula

Critical note. Even with a correct $\bar{p}$, Juan computes V from the *independent-bit* formula $V/M = \frac{1}{2} + \frac{1}{2}(1 - \bar{p})^2$. That expression is ≥ 0.5 for *every* $\bar{p}$ and equals 0.5 only at $\bar{p} = 1$. But the simulation sits *below* 0.5 (0.49 at $\alpha = 2$, falling to 0.47 at $\alpha = 8$). So this formula is on the wrong side of a floor it can never cross, and Juan’s claim that $V \approx 0.55$ “aligns with the simulations” is incorrect — it is in fact *further* from the data than the trivial $\alpha K/2 = 0.5$ of #1.

The reason is the same correlation that Atiyab’s Half B identified: the real system beats 50% violations by *specialising* its variables (your f_j heat-map: variables stable anywhere from 0.25 to 0.96), so AND pairs reach 11 and XOR pairs reach 01/10 *simultaneously*. No single- p formula can represent that; you need the pair quantities $q_{11}^A, q_{\text{eq}}^X$ of (7). Juan went beyond mean-field for p but reverted to mean-field for V .

4.5 Smaller criticisms worth raising on the call

- **Poisson for a bounded count.** The neighbour counts are $\text{Binomial}(\alpha, \cdot)$ (at most α), but Juan models them as *unbounded* Poisson. At the small α we actually use (2–4) this is a poor approximation; the corrected calculation above keeps the exact binomials.
- **The “displaced clause” (FIFO) terms.** His F_{00} includes ± 1 shifts for the clause evicted by the knowledge-base cap. But the greedy decision is evaluated on the KB *after* eviction, so the evicted clause cannot affect whether a flip is accepted; those terms do not belong in an acceptance probability.
- **$p \rightarrow 1$ as $\alpha \rightarrow \infty$ contradicts the data.** Both Juan’s and the corrected mean-field predict $\bar{p} \rightarrow 1$ for large α , but the simulation *saturates* around 0.74. This is *not* fixed by the single-variable correction — it is the independence assumption itself, and only the (unclosed) pair theory can capture it.
- **α bookkeeping.** The note uses $\alpha = 4$; the paper’s main runs use $\alpha = 2$. Any comparison to “the simulation” must fix the same α .

4.6 What is nonetheless right and valuable in Juan’s note

The *strategy* is correct and is the way forward: don’t assume the attractor, compute it from acceptance statistics. His instinct to make the rate degree-dependent for the dynamic case is also right. The execution needs to be rebuilt on the single-variable rules (8)–(9).

5 Synthesis and the real frontier

Contribution	Delivers	Limitation
Atiyab #1	Violation level $V^* = \alpha K/2$ (good to $\sim 2\%$); network-independence of the long run.	Predicts $p = 1$ (false; sim 0.7); right level for the wrong reason.
Atiyab #2	(A) Timescale $\tau = K/2\gamma r$, $r_{\text{int}} \propto$ in-degree (network enters the speed). (B) <i>Exact</i> pair identity for V and <i>exact, single-variable, code-verified</i> flip rules.	(A) rides on the wrong $p = 1$. (B) not closed: pairs need triples.
Juan	The right <i>strategy</i> : compute $\bar{p}$ from acceptance rates rather than assume it.	Built on a two-variable joint flip the model never does; squared prefactors/square root spurious; $\bar{p}$ under-shoots (0.66 vs 0.74); V uses the discredited independent-bit formula and lands on the wrong side of 0.5.

5.1 The through-line

All three rounds keep reusing the single- p , independent-bit picture for V , and **that picture cannot reach the sub-0.5 violation rate the model actually achieves**. The sub-0.5 performance is the whole interesting phenomenon (variables specialise; clauses get jointly satisfied), and it lives in the pair correlations $q_{11}^A, q_{\text{eq}}^X$ — precisely Atiyab’s unclosed pair equations. That is the real frontier.

5.2 Two concrete next steps

1. **Rebuild the steady state on the single-variable rules.** Compute Atiyab’s exact acceptance probabilities $\Pr(A_1 + X_0 > X_1)$ and $\Pr(X_1 > A_1 + X_0)$ (the corrected $F_{\text{up}}, F_{\text{dn}}$ above), giving $\bar{p}$ via (13). We have already done the independent-bit version (table in §4.4); the next refinement is to feed it *correlated* partner statistics rather than independent ones, which is the bridge to closing the pair theory.
2. **Make it dynamic and network-dependent.** Write $\frac{dp_i}{dt} = r_i G(p_i)$ with r_i the (degree-dependent) learning rate and G the corrected single-variable right-hand side. The fixed point $G(\bar{p}) = 0$ is *independent of r_i* , so every agent reaches the same $\bar{p}$ — an analytical statement of the paper’s topology-independence headline — while r_i sets the convergence *speed*, carrying all the transient/network differences. A clean testable prediction is $t_{\text{conv}} \propto 1/(\text{spectral gap})$ with a gap-independent steady state.

A Reproducibility

All numbers were generated by two short scripts in this folder: `_check_juan.py` (measures steady-state p and V/M from the simulation) and `_corrected_mf.py` (solves the corrected single-variable mean-field by exact enumeration of the binomial neighbour counts).